

Scope of Work – Dent Detection Backend

Project: Car Damage Detection & Reporting API

Stack: Flask · PostgreSQL · Cloudinary · YOLO11m model

Date: 18 August 2026

1. Main idea

A user uploads 4–5 photographs of a car. An ML model looks at those photographs, finds the damage, and the system returns a damage report. Every report is saved so the user can look at their past inspections later.

The backend is the middle layer that makes this happen – it takes the images, runs the model, turns the model's output into a readable report, and keeps the history.

2. Damage types detected

The damage types are decided by the model that was supplied, not chosen separately. It detects six kinds of damage:

1. **Dent** – indentations, dings and sheet metal compressions
2. **Scratch** – paint abrasions, scrape lines and clear-coat scuffs
3. **Crack** – windshield fissures and bumper or fender cracks
4. **Glass shatter** – webbed breaks and shattered window panels
5. **Lamp broken** – broken headlight, taillight or turn signal lenses
6. **Tire flat** – deflated tyre, punctured sidewall or exposed rim

Anything outside these six is not reported.

Two points on how this differs from the earlier draft of this document, which listed five damage types of its own choosing:

- **Mirror damage cannot be detected.** The supplied model has no mirror class at all. It has been removed from scope. Adding it later would need the model retrained on labelled mirror images.
- **Hail dents are not separated from ordinary dents.** The model has one general dent class and cannot tell a hail dent from a door ding. Dents are reported simply as dents.

Glass shatter, lamp broken and tire flat are treated as **critical** damage, because they affect whether the car is roadworthy rather than only how it looks. This affects the severity score in section 4.

3. How the project works

Step 1 – Frontend uploads images.

The frontend uploads the car photos straight to Cloudinary. Cloudinary stores the actual image files and returns a link for each one. The backend issues an upload permission first, so the Cloudinary password stays on the server and never reaches the browser.

Step 2 – Frontend sends the links to the backend.

Instead of the image files, the frontend sends the Cloudinary links to the backend, along with optional car details like registration number.

Step 3 – Backend creates an inspection.

The backend saves a new inspection record in the database with the image links and marks it as "in progress". It immediately replies to the frontend with an inspection ID, so the user isn't left waiting on a loading screen.

Step 4 – Model runs on the images.

In the background, the backend picks up each image from Cloudinary and passes it to the model. The model returns what damage it found in that image, where it is, and how confident it is.

Before the model sees a photo, the backend corrects the sideways rotation that phone cameras record in metadata, because the model was trained on upright cars and a rotated photo can produce no findings at all. It also checks whether the photo is too blurry or too dark to judge. If a first pass finds nothing, it automatically retries more sensitively, since faint scratches often sit just below the normal threshold.

Step 5 – Backend builds the report.

The model's raw output is technical, so the backend converts it into something readable – the damage name, where it is on the car, how confident the model is, and how serious it is. Findings from all 4–5 images are then combined into one report for the whole car, with a total count per damage type and an overall severity.

Step 6 – Report is saved and returned.

The report is saved in the database and the inspection is marked complete. The frontend checks back, gets the finished report, and displays it.

4. How severity is decided

The model reports what it found and how confident it is, but not how bad it is. The backend works that out in two ways.

Each individual finding is rated minor, moderate or severe, based on how much of the photo it covers and how confident the model is.

The whole inspection also gets a single **damage score out of 100**, built from how many findings there are, how many of them are critical, and how much of the car's surface is affected. The score is banded into none, minor, moderate or severe for display. Two things are worth knowing about it: the score reaches 100 fairly easily on a badly damaged car, so it ranks light damage against heavy damage but does not separate heavy from very heavy; and the affected-area figure is an average across the photos, not a total, so it stays comparable whether the user sent two photos or five.

All of these thresholds are settings rather than fixed code, so they can be adjusted after seeing real inspections without rebuilding anything.

5. History

Every inspection stays in the database permanently – the car details, the image links, every piece of damage found, the severity, the score, the date, and which version of the model was used.

This means the user can open a list of all their past inspections, filter it by date or damage type, and re-open any old report to see exactly what it said. The model's original output is also kept, so if the report wording or the severity thresholds are changed later, old reports can be rebuilt without running the model again. Deleting an inspection hides it from the user but does not destroy the record.

6. What the backend provides

- Account creation and login
 - Permission for the frontend to upload to Cloudinary
 - Submitting a new set of images for inspection
 - Checking whether an inspection has finished
 - Getting the finished damage report
 - Listing and filtering past inspections
 - Deleting an inspection
 - Damage type list with display colours, for the frontend
 - Summary counts for a dashboard
-

7. Handling problems

- If the model fails, the system retries a few times before marking the inspection as failed.
 - If the model finds nothing, that is a valid result — the report says "no damage detected" rather than showing an error.
 - If a photo was too blurry or too dark to judge, the report says so, so a user with an empty report knows why.
 - If the same submission is sent twice, the second one returns the original inspection instead of creating a duplicate.
-

8. Not included

- Training or improving the model — it is supplied ready-made
 - Any damage other than the six types listed, including mirror damage
-

9. About the model

The model has been supplied and checked. It is an Ultralytics **YOLO11m** detection model trained on a public car damage dataset, and it draws boxes around damage rather than tracing its exact outline. It runs on the ordinary processor, taking a few seconds per image; a graphics card would reduce this to a fraction of a second and is the single biggest speed improvement available.

The backend keeps the model behind a replaceable piece of code, so a retrained or different model can be dropped in later by changing a setting and listing any new damage names, without rewriting the rest of the system.

One limitation to be aware of: the model has no idea what a car is. Given any photograph it will still report damage, and in testing a stock photo of two people produced two confident glass-shatter findings. If users can upload arbitrary images, the reports will sometimes describe damage on things that are not cars.

10. Points to confirm

1. Is the list of critical damage types right — glass shatter, lamp broken and tire flat?
 2. Should the system refuse photos that do not appear to contain a car, or report on whatever it is given?
 3. Is mirror damage needed badly enough to justify retraining the model later?
 4. How long should the photos be kept in Cloudinary?
 5. Will a graphics card be available, or should the system be sized for processor-only speeds?
-

11. Order of work

1. Project setup, database, login
2. Connecting the model and running it in the background
3. Inspection submission, report building, and history
4. Testing and documentation

Remaining before going live: a decision on the points above, real Cloudinary uploads driven from the frontend, and deployment.
